



## Dafny Formal Specification Retrofit — Experiment Complete

**What we built:** A "verified oracle" pipeline where Dafny specs are the single source of truth, and Python conformance is enforced automatically.

```
dafny verify src/dafny/*.dfy          # prove the spec is sound
python scripts/generate_conformance_tests.py # parse .dfy → emit pytest
pytest tests/test_dafny_correspondence.py    # run against y0 Python
```

### By the numbers:

- **4 Dafny specs** (1,192 lines): `probability.dfy` (Kolmogorov axioms), `dag.dfy` (d-separation, semi-graphoid), `do_calculus.dfy` (Rules 1–3, backdoor, frontdoor), `interventional.dfy` (truncated factorization, GlobalMarkov)
- **59 auto-generated conformance tests** across 10 test classes, all passing
- **574 total tests** in the suite, zero regressions
- **~4,300 lines** of new/changed code across 21 files

### What the spec covers (bottom to top):

1. Kolmogorov axioms (non-negativity, normalization, additivity) + derived laws (Bayes, chain rule, total probability, inclusion-exclusion)
2. DAG structure, ancestry, graph surgery ( `RemoveIncoming` , `RemoveOutgoing` )
3. D-separation, semi-graphoid axioms, Local Markov property
4. Pearl's three rules of the do-calculus
5. Backdoor and frontdoor adjustment criteria
6. Markov factorization, truncated factorization ( `TruncatePMF` ), and the grounding of `IntProb` in concrete distributions

### What we learned:

1. **Dafny works well as a verified oracle for an existing Python library.** The parse-and-generate approach means the spec and the tests can't silently diverge — changing a Dafny lemma signature automatically changes the emitted test.
2. **The spec found real bugs.** The original Dafny `BackdoorAdjustment` and `FrontdoorCriterion` used `RemoveIncoming` where they should have used `RemoveOutgoing` . This was masked in Dafny by a `|trail| <= 1` short-circuit in

`TrailBlocked` that silently ignores direct single-step edges, but Python's `are_d_separated` doesn't have that quirk. Fixing the spec to use `RemoveOutgoing` made both implementations consistent with Pearl's textbook formulation. *The math and the implementation were already diverging — the formal spec caught it.*

3. **Symbolic vs. numerical tests serve different roles.** The symbolic DSL can verify algebraic identities (One is multiplicative identity, Zero is absorbing, joint commutativity), but the three core Kolmogorov axioms are inherently numerical. We built a `ConcreteDistribution` class (discrete PMF over a DataFrame) that lets us verify axioms and adjustment formulas on concrete random distributions with fixed seeds.
4. **The "axiom boundary" is clear and honest.** `GlobalMarkov_From_Factorization` (d-separation  $\diamond$  conditional independence) remains `{:axiom}` because a full proof requires Bayes Ball completeness — months of Dafny work. But the axiom boundary is explicit: you can see exactly which claims are proven vs. assumed.
5. **Dafny's type system is both a strength and a limitation.** `real` (arbitrary-precision rationals) avoids floating-point issues, and `map<Outcome, real>` is a clean PMF type. But Dafny has no built-in `product` over a set, so the Markov factorization ( $\prod P(v_i \mid pa(v_i))$ ) had to be axiomatized rather than defined computationally.

## Dafny vs Lean: The Continuous Probability Gap

This experiment stayed entirely in the discrete setting — `PMF = map<Outcome, real>` — and that was deliberate. Dafny's type system fundamentally cannot express continuous probability density functions, and this is worth understanding clearly.

### Why Dafny can't do continuous probability:

Dafny's `real` type is arbitrary-precision *rational*s, not actual reals. It has no concept of limits, completeness, or suprema. This means you cannot define:

- **Sigma-algebras** — the foundation of measure-theoretic probability. You need set operations over uncountable collections, which Dafny's finite `set<T>` can't represent.
- **Lebesgue integration** — PDFs require  $\int f(x) dx = 1$ . Dafny has no integration, no limits, no epsilon-delta.
- **Measurable functions** — random variables in the continuous setting are measurable functions from a probability space to the reals. Dafny has no way to state measurability.
- **Radon-Nikodym derivatives** — a PDF is  $f = \frac{d\mu}{d\lambda}$ , the derivative of one measure with respect to another. This is beyond Dafny's expressiveness.

So if you wanted to formalize " $X \sim \text{Normal}(\mu, \sigma^2)$ " or reason about Gaussian structural equation models, Dafny simply cannot state the theorem, let alone prove it.

### What Lean (Mathlib) already has:

Lean's Mathlib library has formalized most of the infrastructure that Dafny lacks:

| Concept                 | Mathlib                                         | Dafny                                             |
|-------------------------|-------------------------------------------------|---------------------------------------------------|
| Sigma-algebras          | <code>MeasurableSpace</code>                    | —                                                 |
| Measures                | <code>MeasureTheory.Measure</code>              | —                                                 |
| Probability measures    | <code>MeasureTheory.IsProbabilityMeasure</code> | <code>IsDistribution(p)</code><br>(discrete only) |
| Lebesgue integration    | <code>∫ f ∂μ</code>                             | —                                                 |
| PDFs (Radon-Nikodym)    | <code>MeasureTheory.rnDeriv</code>              | —                                                 |
| Conditional expectation | <code>MeasureTheory.conexp</code>               | —                                                 |
| Independence            | <code>ProbabilityTheory.IndepFun</code>         | —                                                 |
| Discrete PMFs           | <code>Pmf</code>                                | <code>map&lt;Outcome, real&gt;</code>             |
| Real analysis / limits  | Full                                            | Rationals only                                    |

In Lean, you could in principle state and prove that the backdoor adjustment formula holds for continuous distributions:

$$P(y \mid do(x)) = \int P(y \mid x, z) P(z) dz$$

In Dafny, we can only verify the discrete sum version:

$$P(y \mid do(x)) = \sum_z P(y \mid x, z) P(z)$$

## Why this didn't matter (yet):

Pearl's do-calculus rules are *structural* — they're about graph topology and d-separation, not about whether distributions are discrete or continuous. The three rules, backdoor criterion, and frontdoor criterion have identical statements in both settings. Our Dafny spec captures the full logical structure:

- D-separation is purely graph-theoretic (no probability needed)
- The rules' *preconditions* are d-separation queries
- The rules' *conclusions* are equalities between interventional distributions

The only place continuous vs discrete matters is when you *ground* `IntProb` in an actual computable distribution — and that's exactly where we used `{:axiom}` boundaries. The `GlobalMarkov_From_Factorization` lemma (d-sep implies conditional independence) is the axiom where the continuous proof would require measure-theoretic machinery that Dafny can't express.

## When it will matter:

If y0 extends to:

- **Continuous SCMs** (linear Gaussian models, nonparametric structural equations) — the truncated factorization involves densities, not PMFs
- **Kernel-based causal inference** — conditional density estimation, RKHS embeddings
- **Semiparametric identification** — functionals of continuous distributions

Then Dafny's discrete PMF approach hits a hard wall. Lean with Mathlib would be the natural choice for those formalizations.

## The tradeoff:

Lean wins on mathematical expressiveness but loses on the code generation pipeline. Dafny can compile to Python, C#, Java, Go, and JavaScript — this is what makes the "verify spec → auto-generate correct implementation" vision feasible. Lean has no comparable code extraction story for Python. You'd need a separate translation layer, which reintroduces exactly the divergence risk we're trying to eliminate.

The pragmatic path: use Dafny for the structural/discrete layer (where it works today), and consider Lean if the project needs to formally reason about continuous distributions. The

axiom boundaries in `interventional.dfy` are exactly where a future Lean formalization would plug in.

### What's next:

- **Implement new identification algorithms directly in Dafny**, verify the proofs, then auto-generate correct-by-construction Python. The current pipeline proves it's viable — the question is whether Dafny's type system can handle the recursive structure of algorithms like ID or IDC\*.
- **Tighten the axiom boundary.** Prove `TruncatePMF_IsDistribution` and `TruncatePMF_Empty` from definitions rather than axioms. Potentially prove `GlobalMarkov` from factorization if the Bayes Ball theorem can be formalized.
- **Extend to ADMGs with latent variables.** The current Dafny spec covers DAGs only. `y0's NxMixedGraph` supports bidirected edges (latent confounders), which need a Dafny representation for the full identification theory.